unie*
Universidad

Grado en Ingeniería Informática

Asignatura: Metodologías de programación
Curso académico: 2025/26
Unidad: 1
Actividad: 1

Planeta Formación y Universidades

Asignatura: Metodologías de programación (Curso 2025/26)
Profesor: Fernando Rodríguez Sela
Título de la actividad: Juego Hundir la Flota

Tabla de Contenido

1 Introducción **3**
1.1 Contexto del juego . . . 3
1.2 Objetivos de aprendizaje . . . 3
1.3 Descripción general . . . 3

2 Desarrollo **4**
2.1 Clase Casilla . . . 4
2.1.1 Atributos . . . 4
2.1.2 Métodos . . . 4
2.2 Clase Barco . . . 4
2.2.1 Atributos . . . 4
2.2.2 Métodos . . . 5
2.3 Clase Tablero . . . 5
2.3.1 Atributos . . . 5
2.3.2 Métodos . . . 5
2.4 Clase Juego . . . 6
2.4.1 Atributos . . . 6
2.4.2 Métodos . . . 6
2.5 Programa principal . . . 7
2.6 Requisitos técnicos . . . 7
2.6.1 Lenguaje de programación . . . 7
2.6.2 Restricciones . . . 8
2.7 Formato de entrega . . . 8

3 Rúbrica de evaluación **9**
3.1 Diseño e implementación de clases (4 puntos) . . . 9
3.2 Funcionalidad (3 puntos) . . . 9
3.3 Validaciones y robustez (1 punto) . . . 9
3.4 Calidad del código (1.5 puntos) . . . 10
3.5 Documentación y entrega (0.5 puntos) . . . 10
3.6 Penalizaciones . . . 10
3.7 Puntos extra (hasta +1 punto) . . . 10

1 Introducción

En esta actividad evaluable se propone el desarrollo de una versión simplificada del clásico juego de mesa **"Hundir la Flota"**. El objetivo principal es aplicar los conceptos fundamentales de la **Programación Orientada a Objetos** (POO) utilizando Java o C++.

1.1 Contexto del juego

El juego "Hundir la Flota" es un juego de estrategia para dos jugadores en el que cada uno coloca una flota de barcos en un tablero oculto. Los jugadores se turnan para realizar disparos sobre las coordenadas del tablero del oponente, intentando hundir todos sus barcos. En esta práctica desarrollaremos una versión simplificada de un solo jugador contra el ordenador.

1.2 Objetivos de aprendizaje

Con la realización de esta práctica se pretende que el alumno:

- Diseñe e implemente clases con atributos y métodos apropiados.
- Aplique los principios de **encapsulación** y **ocultación de información**.
- Utilice **arrays bidimensionales** para representar estructuras de tablero.
- Implemente **composición** y **agregación** entre clases.
- Trabaje con **enumerados** para representar estados.
- Desarrolle métodos de validación y lógica de negocio.
- Implemente un programa principal con interacción por consola.

1.3 Descripción general

El sistema a desarrollar constará de las siguientes clases principales:

- `Casilla`: representa cada posición del tablero con su estado actual.
- `Barco`: representa un barco con su tamaño, posición y estado.
- `Tablero`: representa el tablero de juego con una matriz de casillas y la gestión de barcos.
- `Juego`: coordina la partida, gestiona turnos y determina el fin del juego.

El programa permitirá al usuario jugar una partida completa contra el ordenador mediante un menú interactivo por consola.

2 Desarrollo

2.1 Clase Casilla

La clase `Casilla` representa cada posición individual del tablero. Debe contener:

2.1.1 Atributos

- `estado`: un tipo enumerado que puede tomar los valores:
  - `AGUA`: casilla sin barco y sin disparar.
  - `BARCO`: casilla con barco y sin disparar.
  - `TOCADO`: casilla con barco que ha sido disparada.
  - `AGUA_DISPARADA`: casilla sin barco que ha sido disparada (agua).

2.1.2 Métodos

- Constructor que inicialice la casilla en estado `AGUA`.
- `getEstado()`: devuelve el estado actual de la casilla.
- `setEstado(nuevoEstado)`: modifica el estado de la casilla.
- `tieneBarco()`: devuelve `true` si el estado es `BARCO` o `TOCADO`.
- `disparar()`: realiza un disparo sobre la casilla y actualiza su estado apropiadamente. Devuelve `true` si se ha tocado un barco.
- `toString()`: representación en forma de carácter de la casilla para mostrarla por pantalla (por ejemplo: '~' para agua, 'B' para barco, 'X' para tocado, 'O' para agua disparada).

2.2 Clase Barco

La clase `Barco` representa un barco en el tablero. Debe contener:

2.2.1 Atributos

- `tamano`: número entero que indica el tamaño del barco (número de casillas que ocupa).
- `orientacion`: tipo enumerado que puede ser `HORIZONTAL` o `VERTICAL`.
- `filaInicio`: fila de la primera casilla del barco.
- `columnaInicio`: columna de la primera casilla del barco.
- `impactosRecibidos`: contador de impactos que ha recibido el barco.

2.2.2 Métodos

- Constructor que reciba el tamaño, orientación, fila y columna inicial.
- `getTamano()`: devuelve el tamaño del barco.
- `getOrientacion()`: devuelve la orientación del barco.
- `getFilaInicio()` y `getColumnaInicio()`: devuelven las coordenadas de inicio.
- `registrarImpacto()`: incrementa el contador de impactos recibidos.
- `estaHundido()`: devuelve `true` si el número de impactos es igual al tamaño del barco.
- `ocupaCasilla(fila, columna)`: devuelve `true` si el barco ocupa la casilla indicada.

2.3 Clase Tablero

La clase `Tablero` representa el tablero de juego completo. Debe contener:

2.3.1 Atributos

- `FILAS`: constante con el número de filas del tablero (recomendado: 10).
- `COLUMNAS`: constante con el número de columnas del tablero (recomendado: 10).
- `casillas`: matriz bidimensional de objetos `Casilla`.
- `barcos`: array o lista de objetos `Barco`.
- `numeroBarcos`: contador de barcos en el tablero.

2.3.2 Métodos

- Constructor que inicialice la matriz de casillas y el array de barcos vacío.
- `colocarBarco(tamano, fila, columna, orientacion)`: intenta colocar un barco en la posición indicada. Debe validar que:
  - Las coordenadas sean válidas.
  - El barco no se salga del tablero.
  - No se solape con otros barcos.
  - Haya al menos una casilla de separación con otros barcos (opcional).

  Devuelve `true` si se ha colocado correctamente.
- `colocarBarcosAleatoriamente()`: coloca automáticamente una flota predefinida de barcos en posiciones aleatorias válidas. La flota debe contener:
  - 1 barco de tamaño 4 (portaaviones)
  - 2 barcos de tamaño 3 (acorazados)
  - 3 barcos de tamaño 2 (destructores)

  - 4 barcos de tamaño 1 (lanchas)

- `recibirDisparo(fila, columna)`: procesa un disparo en las coordenadas indicadas. Debe:
  - Validar que las coordenadas sean correctas.
  - Verificar que no se haya disparado previamente en esa casilla.
  - Actualizar el estado de la casilla.
  - Si hay un barco, registrar el impacto correspondiente.
  - Devolver un código o mensaje indicando el resultado (agua, tocado, hundido).
- `todosLosBarcosHundidos()`: devuelve `true` si todos los barcos han sido hundidos.
- `mostrarTablero(ocultarBarcos)`: muestra el tablero por consola. Si el parámetro es `true`, no muestra los barcos que no han sido tocados (para el tablero del oponente).
- `esCoordenadasValida(fila, columna)`: valida que las coordenadas estén dentro del tablero.

2.4 Clase Juego

La clase `Juego` coordina la lógica de la partida. Debe contener:

2.4.1 Atributos

- `tableroJugador`: objeto `Tablero` para el jugador humano.
- `tableroOrdenador`: objeto `Tablero` para el ordenador.
- `turnoJugador`: booleano que indica de quién es el turno.
- `numeroTurnos`: contador de turnos realizados.
- `juegoTerminado`: booleano que indica si el juego ha finalizado.

2.4.2 Métodos

- Constructor que inicialice los dos tableros y coloque los barcos aleatoriamente en ambos.
- `iniciarJuego()`: método principal que ejecuta el bucle del juego hasta que termine.
- `turnoDelJugador()`: gestiona el turno del jugador:
  - Muestra el tablero del ordenador (con barcos ocultos).
  - Solicita coordenadas de disparo al usuario.
  - Procesa el disparo y muestra el resultado.
- `turnoDelOrdenador()`: gestiona el turno del ordenador:

  - Genera coordenadas aleatorias de disparo.
  - Procesa el disparo en el tablero del jugador.
  - Muestra el resultado.

- `verificarFinDeJuego()`: comprueba si alguno de los dos tableros tiene todos los barcos hundidos y actualiza el estado del juego.
- `mostrarResultadoFinal()`: muestra el resultado final de la partida (ganador y estadísticas).

2.5 Programa principal

El programa principal debe:

1. Mostrar un mensaje de bienvenida con las reglas básicas del juego.
2. Crear una instancia de la clase `Juego`.
3. Iniciar la partida mediante el método `iniciarJuego()`.
4. Al finalizar, preguntar si se desea jugar otra partida.

2.6 Requisitos técnicos

2.6.1 Lenguaje de programación

El alumno puede elegir entre **Java** o **C++** para la implementación. Los requisitos específicos para cada lenguaje son:

Java:

- Utilizar tipos enumerados (`enum`) para estados y orientaciones.
- Utilizar arrays estáticos para almacenar los barcos.
- Utilizar la clase `Scanner` para la entrada de datos.
- Utilizar `Random` para generar coordenadas aleatorias.
- Todas las clases deben estar en archivos separados.

C++:

- Utilizar `enum` para estados y orientaciones.
- Utilizar arrays estáticos para almacenar los barcos.
- Utilizar `printf` y `scanf` o `cin` y `cout` para entrada/salida.
- Utilizar `rand()` de la librería `math` para generar coordenadas aleatorias.
- Separar declaración (.h) e implementación (.cpp) de las clases.

2.6.2 Restricciones

- **No se permite el uso de herencia.** Todos los conceptos deben implementarse mediante composición y agregación.
- No se permite el uso de librerías gráficas o interfaces gráficas. Todo debe funcionar por consola.
- El código debe estar correctamente comentado, especialmente los métodos más complejos.
- Se debe aplicar encapsulación: los atributos deben ser privados y el acceso debe realizarse mediante métodos públicos.

2.7 Formato de entrega

La práctica se entregará en un archivo comprimido (.zip o .tar.gz) que contenga:

- Todos los archivos de código fuente (.java o .cpp/.h).
- Un archivo `README.txt` con:
  - Nombre y apellidos del alumno.
  - Instrucciones de compilación y ejecución.
  - Breve descripción de la estructura del proyecto.
  - Decisiones de diseño tomadas.

3 Rúbrica de evaluación

La práctica se evaluará sobre **10 puntos** según los siguientes criterios:

3.1 Diseño e implementación de clases (4 puntos)

- **Clase Casilla (0.75 puntos)**
  - 0.25 puntos: Enumerado correcto y atributos adecuados.
  - 0.25 puntos: Métodos correctamente implementados.
  - 0.25 puntos: Método `disparar()` con lógica correcta.
- **Clase Barco (1 punto)**
  - 0.25 puntos: Atributos correctos y constructor.
  - 0.25 puntos: Método `ocupaCasilla()` correcto.
  - 0.25 puntos: Método `estaHundido()` correcto.
  - 0.25 puntos: Gestión correcta de impactos.
- **Clase Tablero (1.5 puntos)**
  - 0.3 puntos: Inicialización correcta de la matriz y arrays.
  - 0.4 puntos: Método `colocarBarco()` con validaciones completas.
  - 0.4 puntos: Método `colocarBarcosAleatoriamente()` funcional.
  - 0.4 puntos: Método `recibirDisparo()` con lógica correcta.
- **Clase Juego (0.75 puntos)**
  - 0.25 puntos: Inicialización correcta de tableros.
  - 0.25 puntos: Turnos implementados correctamente.
  - 0.25 puntos: Verificación de fin de juego correcta.

3.2 Funcionalidad (3 puntos)

- **1 punto**: El juego se ejecuta sin errores y permite jugar una partida completa.
- **0.75 puntos**: La colocación automática de barcos funciona correctamente y cumple todas las restricciones.
- **0.75 puntos**: Los disparos se procesan correctamente (agua, tocado, hundido).
- **0.5 puntos**: El juego detecta correctamente el final y muestra el ganador.

3.3 Validaciones y robustez (1 punto)

- **0.4 puntos**: Validación correcta de coordenadas de entrada.
- **0.3 puntos**: Gestión de errores de entrada (valores no numéricos, fuera de rango).
- **0.3 puntos**: Prevención de disparos repetidos en la misma casilla.

3.4 Calidad del código (1.5 puntos)

- **0.4 puntos**: Encapsulación correcta (atributos privados, métodos públicos apropiados).
- **0.3 puntos**: Nombres de variables y métodos descriptivos.
- **0.3 puntos**: Código correctamente indentado y estructurado.
- **0.3 puntos**: Comentarios adecuados en código complejo.
- **0.2 puntos**: Sin código duplicado, métodos con tamaño razonable.

3.5 Documentación y entrega (0.5 puntos)

- **0.25 puntos**: Archivo README completo con instrucciones claras.
- **0.25 puntos**: Estructura de archivos organizada, compilación sin problemas.

3.6 Penalizaciones

- **-0.5 puntos**: Entrega fuera de plazo (por cada día de retraso, máximo -2 puntos).
- **-0.5 puntos**: Código que no compila.
- **-0.25 puntos**: Uso de librerías no permitidas.

3.7 Puntos extra (hasta +1 punto)

- **+0.5 puntos**: Implementación de una IA básica para el ordenador que no dispare aleatoriamente (por ejemplo, que continúe disparando alrededor cuando toca un barco).
- **+0.3 puntos**: Permitir al jugador colocar manualmente sus propios barcos al inicio de la partida.
- **+0.2 puntos**: Implementar un sistema de estadísticas (precisión de disparos, barcos hundidos por tipo, etc.).